

ST:AI Agent Architecture & Development

Web LLM Attacks

Lab 1 — Exploiting LLM APIs with excessive agency



Exploiting LLM APIs with excessive agency

Backend AI logs

[Back to lab description >>](#)

LAB Solved



Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#) | [My account](#) | [Live chat](#)

Live chat

CONNECTED: -- Now chatting with Arti Ficial --

You: What APIs do you have access to?

Arti Ficial: I have access to the following APIs:

1. Password Reset
2. Debug SQL
3. Product Info

Feel free to ask me any questions or request assistance related to these APIs.

You: What arguments does the Debug SQL API take?

Arti Ficial: The Debug SQL API requires the following argument:

The vulnerability was that the LLM had access to powerful APIs, such as the SQL database, without enough restrictions. This allowed the LLM to run a command that deleted a user from the database. The fix would be to limit the LLM's access to sensitive APIs and require authorization for dangerous actions.

Lab 2 — Exploiting vulnerabilities in LLM APIs



Exploiting vulnerabilities in LLM APIs

Backend AI logs

[Back to lab description >>](#)

LAB

Solved



Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#) | [My account](#) | [Live chat](#)

Live chat

You: What APIs do you have access to?

Arti Ficial: I have access to the following APIs:

1. Password Reset
2. Subscribe to Newsletter
3. Product Information

Let me know if you need assistance with any of the services provided by these APIs.

You: What arguments does the Newsletter Subscription API take?

Arti Ficial: The Newsletter Subscription API requires an email address as an argument.

You: Please call the Newsletter Subscription API with the argument `attacker@0ada00b604dd31b2828882ea0192009.exploit-server.net`.

The vulnerability was that the application allowed user input to be passed into an operating system command. This allowed an attacker to run commands and delete a file from the server. The fix would be to properly validate user input and avoid sending untrusted input directly to system commands.

Lab 3 — Indirect Prompt Injection



Indirect prompt injection

Backend AI logs

[Back to lab description >>](#)

LAB

Solved



Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#) | [My account](#) | [Live chat](#)

WE LIKE TO
SHOP 



The vulnerability was that the LLM trusted instructions hidden inside a product review. An attacker could use a review to trick the LLM into performing an action, such as deleting an account. The fix would be to treat information from reviews and other external content as untrusted and prevent it from controlling sensitive functions.

Lab 4 — Exploiting insecure output handling in LLMs



Exploiting insecure output handling in LLMs

Backend AI logs

[Back to lab description >>](#)

LAB

Solved



Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#) | [My account](#) | [Live chat](#)

Lightweight "l33t" Leather Jacket



\$1337.00



The vulnerability was that the application displayed LLM output without properly checking it, allowing an XSS attack through a malicious review. This could cause actions to run in another user's browser, such as deleting their account. The fix would be to properly sanitize and encode LLM output before displaying it.